答辩知识手册

四六级词汇量检测 · 面对专业评委的问答准备 · 共 6 章 / 70 问

用法：先看第五章"三层话术"和第六章"应急清单"，再按评委可能在的方向翻对应章节。

一、前端工程（框架取舍 / 渲染 / 状态 / 存储 / 构建）
二、UI 设计（令牌 / 栅格 / 排版 / 色彩 / 组件 / 动效 / 无障碍）
三、统计与测量（抽样 / 区间 / 蒙对 / 信度效度）
四、AI 与后端（工具调用 / RAG / SSE / 安全 / 降级）
五、小程序（双线程 / rpx / setData / 复用）
六、产品与现场（差异化 / 商业化 / 应急 / 三层话术）

一、前端工程

1.1 框架取舍

评委可能问	你这样答（30 秒）
为什么不用 Vue / React?	因为这个产品的交付形态是"发一个 HTML 文件给对方，双击就能用、断网也能用"。框架会带来构建产物、运行时、路由这一整套东西，装不进去一个 2 MB 的单文件。反过来我用原生写了 8 个模块 + 一套构建脚本，工程上并不比框架松；439 条断言和可复现构建就是证据。
没有组件化，代码怎么维护?	有组件，只是不是框架组件：卡片、胶囊、开关、条形进度、按钮变体都是 CSS 类 + 一个模板函数（`tpl()`）拼出来的；页面是 `viewXxx()` 返回 HTML 字符串，事件统一走 `data-act` 委托。换成框架，这层的职责划分是一模一样的。
渲染为什么用字符串拼接而不是虚拟 DOM?	整页重渲染一次约 1–3 ms，页面元素在几百个量级，虚拟 DOM 的 diff 成本反而更高。这种"页面级重渲染"的好处是永远不会出现"状态改了但界面没同步"的 bug——这也是我 439 条断言里很多能直接断言最终 DOM 的原因。
事件怎么绑定？会不会内存泄漏?	一个事件委托挂在根容器上，按 `data-act` 分发（`A.bind`），所以整页重渲染不会重复绑定、也不会漏绑定。定时器由各自模块持有句柄并在切题/退出时清理，`beforeunload` 里做一次 save。

1.2 状态与存储

评委可能问	你这样答
状态管理是什么模型?	单一状态对象 `S`：一份 JSON，改了之后调 `A.save()` 落盘、`A.render()` 重渲染。没有中间层，也没有不可变数据结构——因为全部数据加起来不到 1 MB，重渲染一次几毫秒，没必要引入复杂度。
数据量多大？会不会超过 localStorage 限制?	作答状态每个词 1 字节（0/1/2 + 缺失 = 未测），7,175 词约 7 KB；每轮归档再用 2 bit/词 压缩成位图，约 1.8 KB，最多留 12 轮。总量在百 KB 级，localStorage 5 MB 的上限有一到两个数量级的余量。
为什么用位图?	跨轮对比要逐词比较，JSON 存 7,175 个键约 40 KB/轮，位图只要 1.8 KB，12 轮下来差 20 倍。2 bit 刚好能表示 4 种状态（未测 / 不认识 / 模糊 / 认识）。
词库升级了怎么办?	每轮归档会存一份"词表指纹"（FNV-1a，形如 `7175:xxxx`）。指纹对不上就只做聚合对比（词汇量、七段），不硬算逐词差异——否则会把"两套词库"的差异误当成"背了又忘"。
多用户怎么隔离?	使用者档案：第一个档案沿用老键 `cet4vocab_v1`（兼容旧数据），其余是 `cet4vocab_v1::<id>`，外带一份档案表 `cet4vocab_profiles`。切换档案是"就地换装"——保持 `A.S` 引用不变，只换内容，所有模块不用改。
浏览器禁用了本地存储呢?	启动时探一次，失败就置 `A.storageBroken`，界面提示"本次数据无法保存"，其余功能照常——不因为存储不可用就白屏。

1.3 性能与工程化

评委可能问	你这样答
首屏多少？怎么优化到这一步的?	本机 DOMContentLoaded 约 120 ms。主要手段：零框架、零网络请求（全部内联）、渲染只做一次 innerHTML、动效只用 transform/opacity（不触发布局重排）、入场页盖在已渲染好的主界面上（退场是纯视觉，没有加载空窗）。
2 MB 一个文件，手机上打开慢不慢?	文件是本地读的（不进网络），解析 2 MB JS + 词库约 200–400 ms，之后全部在内存里。代价是首次下载 2 MB，换来的是"永远不再请求任何资源"。
构建怎么做？会不会不可复现?	一个 Node 脚本把 CSS/JS/词库/例句/立绘按固定顺序拼进模板。产物**不写构建时间戳**，同样的源码构建出的文件逐字节相同（可以比 SHA256）。这条规则重要，因为"发出去的到底是哪一版"必须能说清。
测试怎么做的?	四层：jsdom 跑功能（207 条）、真实 Chrome 跑布局与对比度（55 条）、真实 Chrome 跑桌宠物理（40 条）、真机宽度下的孤立单文件验收（20 条）。抽样与区间估计那部分有 60 多条断言，改一个参数就挂。
怎么保证"发给别人也能用"?	专门有一套"收件人环境"测试：空目录 + 全新浏览器配置 + 屏蔽所有 http/https，跑 20 项断言（能不能答题、进度是否落盘、手机布局、顶栏单行）。这条最容易出事故，所以我把它做成自动化。

二、UI 设计（这是"美化"背后的知识）

先记住一句话

评委问"你的设计思路是什么"，不要答"我觉得这样好看"。要答：**先定令牌，再定层级，最后才谈具体页面**——这也是专业设计流程的顺序。

2.1 设计令牌（Design Tokens）

设计令牌就是"把设计决定变成变量"，让同一套视觉规则能在浅色/深色、网页/小程序里复用。我们的令牌大致是：

色彩	语义色而非"红黄蓝"：`--ok`（认识/绿）、`--warn`（模糊/琥珀）、`--bad`（不认识/红）、`--accent`（品牌蓝）。语义色的好处是深色模式只换值、不换用途。
文字三级	`--ink` 主文字 / `--ink2` 次要 / `--ink3` 弱化。所有正文只用这三档，避免"随手取灰度"导致层级混乱。
表面与描边	`--glass`（卡片底）、`--line`（1px 分隔）、`--line2`（强调描边）。
圆角与阴影	卡片 16–20px、按钮 12–14px、胶囊 999px；阴影只用单层低透明度（`0 20px 48px -18px` 这种负扩散写法），保证"浮起但不脏"。

2.2 间距与栅格

知识点	我们怎么用
8pt 栅格	所有间距取 4 的倍数（4/8/12/16/24/32）。好处是放大缩小都对齐，不会出现"这里差 3px"的别扭。
页面留白	桌面容器最大 1000–1100px 居中；左右内边距 24px；卡片间隔 16–22px；内容区不跑满整宽（阅读行长控制在 60–75 个字符）。
层级节奏	页面标题 27–30px / 区块标题 14px + 等宽小字副标 / 正文 13–13.5px / 小字 11–12.5px。**标题大、正文稳、副标弱**，三层就够。
为什么不用 12 栏栅格	这不是营销页，页面结构是"卡片流 + 侧栏"，用 Flex + 最大宽度约束比 12 栏更省事；栅格的价值在多列对齐，我们大多是单列卡片。

2.3 色彩与对比度（评委最爱问）

评委可能问	你这样答
你的对比度达标吗?	实测：辅助文字 **4.83:1**、开关轨道 **5.72:1**，都过了 WCAG AA（正文 4.5:1、控件 3:1）。这两个数是浏览器里真实算出来的，写在自动化断言里，不是设计稿上标的心愿。
为什么强调"语义色"?	因为同一颜色在不同位置含义不同：绿不只表示成功，还表示"认识"这一掌握状态。语义化之后，深色模式只需重新映射数值，含义不变；换主题不会把"认识"变成"警告"。
深色模式怎么保证一致?	同一份令牌按 `html[data-theme="dark"]` 重新赋值，所有组件自动跟随。曾经踩过坑：全局 button 规则上加了背景过渡，切换主题时旧底色被锁住 → 卡片变"浅底浅字"，后来去掉过渡并加断言。

2.4 组件化（"组件美化"具体是什么）

我们手写的组件其实是一套"受控的视觉零件"，每个零件都有固定规则：

组件	设计规则
按钮	三档：主（品牌色实底）/ 次（浅底）/ 幽灵（描边）。高度统一（桌面 40px、手机 44px 起），点击态统一为 `scale(.985) + opacity .92`，不写花哨 hover。
卡片	白底 + 1px 描边 + 单层阴影 + 16–20px 圆角；卡片内部固定"标题 / 说明 / 操作"三段，操作永远在底部对齐。
胶囊 / 徽标	999px 圆角，字号 11–12px，语义色浅底 + 深字。用于状态（认识/模糊/不认识）、等级（S/A/B+）、档位（本地方案）。
条形与环	条形高度 6–12px、圆角端点；环形用 SVG stroke-dasharray 画进度。两者都只用颜色编码，不靠文字解释。
开关	44×24px 轨道 + 20px 圆钮，`role="switch"` + `aria-checked`；键盘可用，深色模式重算轨道色（保证 3:1）。
空态 / 加载态 / 错误态	每个页面都准备了三态：没数据给"下一步点哪里"（不是空白）；AI 等待用点点点动画；连线失败降级成黄色提示条 + 自动回落，而不是红字报错。

2.5 动效原则

知识点	我们怎么用
为什么只动 transform / opacity?	这两个属性由合成器处理，不触发布局重排和重绘，低端机也不掉帧。动 `width/height/top/left` 会触发 layout，长列表上必卡。
时长与曲线	进场 ≤300ms、悬停 150–200ms、入场页总时长约 1.1s + 0.52s 退场；曲线用 ease-out 家族，收尾不拖。
尊重系统设置	`prefers-reduced-motion` 下关掉所有动画，但**入场页仍然存在、仍要点击**——尊重设置不等于替用户做决定。
动效的目的是什么?	三件事：交代空间关系（卡片从哪来到哪去）、反馈操作（按下缩放）、制造节奏（数字滚动）。不为了炫技加动效。

2.6 无障碍（专业评委常问，也是加分项）

键盘能用吗?	能：答题页 1/2/3/4 作答、空格或回车下一题、Backspace 撤销、Esc 退出；入场页打开即聚焦"进入"，进入后焦点交还主内容区 `#view`（`tabindex=-1`）。
模态与入场页怎么处理焦点?	打开时给底层加 `inert`——键盘 Tab 和读屏都进不去不可见的内容，这是比"手动禁 Tab"更可靠的做法。
读屏能读吗?	关键操作有 `aria-label`（含品牌标识 `aria-label="30 · 四六级词汇量检测"`）、状态用 `role="switch"` + `aria-checked`；答题时通过 `aria-live` 播报"已标记为认识"。
还有哪些无障碍细节?	减少动效支持、不依赖颜色单独传达信息（百分比同时给数字）、触控目标 ≥44px、文本框有 visible label。做不到的会老实说：暂不支持屏幕阅读器全流程。

三、统计与测量（最容易被问穿的一章）

3.1 抽样

评委可能问	你这样答
你的抽样方法叫什么?	按词频分层的等比例随机抽样（stratified sampling，比例分配）。总体是有限的（4,540 个词）、抽样不放回，所以用有限总体修正。
为什么分层?	词汇掌握率在频段之间差异极大（高频词认识、冷门词不认识），段内差异小。分层之后"这次抽到几个冷门词"的运气成分被压掉：同样 100 题，分层区间 ±62 词，简单随机 ±91 词。
为什么是"交错抽"而不是"先把一段抽完"?	因为要支持中途停。交错抽取保证**任何前缀都是一个合法的等比例分层样本**——这是"边测边算精度、达标就停"在统计上成立的前提。先抽完一段的话，停在中途就只是一个偏样本。
顺序为什么可复现?	用词自身的 FNV-1a 哈希定序：视觉上完全打乱，但同一台机器、每一次打开顺序都一样。好处是"第 3 组"永远是同一批词，跨轮对比才有意义。
样本量怎么定?	不是固定值，是"目标精度反推"：给定目标半宽 h，`n ≈ p(1-p) (1.96/half_word)^2 × 0.7`（分层折扣），再按实际作答动态修正，达标即停。

3.2 估计与区间

```
段内掌握率      p_b = (k_b + 0.5·f_b) / n_b          折中口径（严格口径只算 k）
整库掌握率      p   = Σ(N_b · p_b) / N
词汇量          vocab = p · N
区间半宽（词）  half = 1.96 · sqrt(max(p(1-p), 1e-6) / n) · sqrt((N-n)/(N-1)) · N
```

三层修正是哪三层?	① Agresti-Coull（小样本/全对全错时不让区间塌陷）；② 有限总体修正 `sqrt((N-n)/(N-1))`（不放回抽样）；③ 蒙对校正（四选一按 `(p-1/k)/(1-1/k)` 扣掉 25% 蒙对）。另设方差下限 1e-6 兜底。
为什么方差要有下限?	p=0 或 p=1 时 p(1-p)=0，公式会算出"误差为零"——那是在撒谎。加 1e-6 的下限，保证永远给出一个非零区间。
"95% 区间"怎么解释?	不是"真值有 95% 概率落在这里"，而是"重复抽样很多次，约 95% 的区间会覆盖真值"。所以界面上写的是"重复抽样时约 95% 会盖住真值"，没写成承诺。
两套口径为什么都要给?	因为"模糊"这一档是主观的。严格口径只算认识（偏保守），折中口径把模糊算一半（偏乐观）。两个数并列，用户自己判断，我不替他挑一个好看的。
覆盖率到 95% 之后呢?	直接转普查（census）：逐词统计，不再做抽样估计，报告里写"实测值、无抽样误差"。这是把"估计"平滑过渡到"枚举"。

3.3 测量的诚实边界（主动说，别等人问）

你的信度（test-retest）是多少?	老实说：**没有做大样本信度研究。**我给的是工程层面的可验证性（同一份数据结果完全可复现）+ 统计层面的区间。要做信度需要同一批人隔两周重测，这是下一步。
效度呢？和四级分数相关吗?	还没做相关分析。逻辑上它测的是"大纲词汇的识别率"，和四级阅读的相关性应该不低，但没有数据前我不下结论——这正是我不把等级叫"四级预测分"的原因。
自评会不会自欺?	会。缓解手段有三个：双口径并列、四选一模式扣蒙对、以及"复习效应提示"（本次抽到此前测过的词超过 20% 时，结果旁边标注可能偏高）。
最小可检测变化（MDE）是多少?	两次测量的差异若要算"真实变化"，至少要超过 ±区间半宽。所以界面上写"差值在 ±436 的抽样误差以内，先别当成进步"——不把噪声说成进步。
为什么不按缺口大小排补词顺序?	因为一个词的价值和它的出现频率大致成正比。10k-20k 段可能缺得最多，但那些词一年遇不上几次；最高频段缺几个词，却是每次阅读都撞上的。所以按「缺口词数 × 词频权重」排序，权重从 1 递减到 0.02。

四、AI 与后端

4.1 AI 的设计原则（评分表 35% 就在这里）

评委可能问	你这样答
AI 在你的产品里具体做什么?	只做三件事：**查数据、解释数据、给动作。**所有数字由本机的统计代码算出，模型不能自己算——它想回答"该补哪一段"，必须走工具取频段数据，数字由工具返回。
工具调用怎么实现的?	一个 40 行左右的循环：模型返回 tool_calls → 服务端执行工具 → 把 JSON 结果回灌 → 最多 3 轮。8 个只读工具（整体情况、频段、缺口词、计划、例句、轮次差分、地图格、额度）+ 1 个动作工具 suggest_actions。
为什么手写不用 LangChain / Vercel AI SDK?	需求很窄：8 个工具、3 轮上限、SSE 六类事件。手写循环更好调试、零依赖，功能等价。如果团队更大、工具更多，我会换成框架。
怎么防止模型乱说?	三条硬规矩写进 System Prompt 并断言：① 数字必须来自工具；② 数据不足（样本 <30 或覆盖 <95%）直接说"下不了结论"；③ 口径只有严格/折中两种，不许自创。另外回答上方会标出这次调了哪些工具，下方能展开"本次发送给模型的数据"。
模型输出被当可信的吗?	不。动作必须过白名单（go / start / review / ask）：路由要合法、target 夹到 {150,250,400}、频段夹到 0–6、问题截断 120 字。否则模型一句"删掉数据"就可能真执行。

4.2 RAG 与例句

你的 RAG 是怎么做的?	轻量方案：9,204 句英中对照语料（Tatoeba，CC-BY 2.0 FR），构建期建"单词 → 候选句"倒排索引，并按可读性打分成（词数接近 10–12、长度 60–80 字符、主语开头加分）；运行期模型按需调 `get_examples` 取真实例句。
为什么不用向量检索?	向量方案要拖几十 MB 模型权重，直接毁掉"单文件 + 断网可用"。而这里的检索需求是"精确命中某个单词的例句"，倒排索引又快又可解释，还能断言。
怎么防止 AI 编例句?	例句永远从语料里查，不生成；界面每条例句标来源（Tatoeba），可溯源。

4.3 后端与流式

后端是什么技术栈?	Node 原生 `http`，零 Web 框架，约 400 行：静态托管 + Agent 循环 + 工具层 + 语料检索。生产依赖 0 个。
流式怎么实现?	SSE（Server-Sent Events）：服务端按行推事件，前端用 fetch + ReadableStream 逐块解析。事件类型六种：`mode`（当前档位）、`tool`（调了哪个工具）、`delta`（文字增量）、`actions`（可点动作）、`done`、`error`。所以"工具调用可视化"是协议设计的自然结果，不是事后补的动画。
为什么用 SSE 不用 WebSocket?	这是单向推送场景（服务端 → 浏览器），SSE 一行 `text/event-stream` 就够，还能自动重连；WebSocket 需要额外的协议与心跳，收益为零。
模型 Key 怎么保护?	公开仓库产物里**没有任何 Key**（有脚本 `check-no-key.mjs` 守着，明文 `sk-` 出现即失败）；评委版才内联作者 Key。可选的授权锁是 Key 用 `PBKDF2-SHA256（60 万次）+ AES-256-GCM` 加密后内联，解出来只存内存、刷新失效。
额度怎么管?	状态机：完成一次有效检测换 1 次（整库要 ≥100 词且覆盖 ≥95%，摸底要 ≥80 题且达标）；同一轮同口径用 `kind:bank:round.startedAt` 去重，防止刷；扣减有下限保护；用尽自动降级到本地方案并说明。
多个模型怎么适配?	一层适配器：DeepSeek / 通义千问 / 智谱（OpenAI 兼容协议）+ 本地 Ollama，改 `.env` 里 `LLM_PROVIDER` 一行切换；没配置就走 mock 本地方案。跨域实测：DeepSeek 允许浏览器直连，OpenAI/智谱默认拒绝。

五、小程序

评委可能问	你这样答
小程序和网页版是什么关系?	同源引擎、两套视图。统计引擎按"注入依赖"的方式移植（存储换成本地适配层、词库改为模块引入），用 **1,302 条字段级断言** 和 **34,310 条答题流程断言** 保证两端逐字段一致。
为什么要两套视图?	交互模型不同：网页是键盘 + 大屏，小程序是触屏 + 单列。我保留算法、重写界面，而不是把网页塞进 webview——那样在触屏上又慢又别扭。
小程序的双线程模型是什么?	逻辑层（JS，跑在 JSCore/V8）和渲染层（WebView）分离，两层通过 `setData` 通信，数据要序列化后传过去。所以 `setData` 越大越慢，原则是"只传变化、不传大对象"。
rpx 是什么?	响应式像素：750rpx = 屏幕宽度。设计稿按 750 宽出，写 rpx 就能等比缩放，不需要自己算 dpr。
主包大小限制?	单包 2 MB。现在主包 1.58 MB（词库 824 KB + 例句 585 KB + 代码），没有分包。
小程序上哪些功能没做？为什么?	发音要 TTS 插件、AI 要 request 合法域名或云函数、词汇地图 26 列在 375pt 上每格不到 14rpx 看不清。这些是取舍，不是遗漏——小程序端先保"测准 + 诊断 + 补词"这条主链。

六、产品、差异化与现场

6.1 差异化（三句能背的）

和百词斩/扇贝的词汇量测试有什么不同?	三个层面：① 它给一个数，我们给"数 + 95% 区间 + 两套口径"；② 它测完推背词，我们测完给"先补哪一段、哪些词、几天补完"；③ 它要注册要联网，我们一个文件、断网可用、数据只在本机。
好在哪？能不能当场证明?	能，两条：**同一份数据两次跑结果完全一样**（哈希定序）；**全答对时区间不塌成 0 宽**（Agresti-Coull + 方差下限）。这两条别人做不出来或没做。
不如别人在哪?	只测"认不认识"，不测听写说；没有大样本常模和信效度检验；绝对词数不能跨工具比较；移动端和分发远不如成熟 App。
商业化怎么想?	不做账号、不导流、不卖数据。可能的收入在 B 端：教师/机构版（班级汇总）、学校授权与定制词库、测评人次分成。免费部分永远是完整的测量与诊断，不阉割核心。

6.2 过程与反思（评委很吃这一套）

最大的失败是什么?	入场页的 favicon 用正则截断了 data URI，导致页面凭空多出一行乱码 `>3" >`；修完之后我加了"启动时清理游离文本节点"的兜底。教训：用正则处理 data URI 必须先转义、按整行替换。
有没有测试骗过你?	有。浏览器断言脚本里一处跨行单引号造成语法错误，而脚本只负责生成页面并打印 ok，所以**断言一条都没跑却一直是绿的。**后来补了自动跑分器（有 FAIL 就非 0 退出），并养成"看数字来源"的习惯。
如果重做会怎么改?	三件：① 把"每日可投入时间"这类跨模块参数从第一天就收敛成单一函数（我犯过两次写死 50 的错）；② 测试先写"收件人环境"（别人电脑上能不能用）；③ 数据层与视图层更早分离，小程序移植会更省力。
一个人怎么分工?	按"数据 → 引擎 → 视图 → 测试"四层推进，每层做完立刻写断言再动下一层。文档（NEXT.md）当外部记忆用：每个决策记"为什么这么选、什么路试过不行"，避免自己重复踩坑。

6.3 三层话术（按场合挑）

`20 秒电梯版`

"我做的是一个词汇量**测量工具**，不是背单词软件。用按词频分层的随机抽样，3 分钟给出带 95% 区间的词汇量、七段掌握结构和一份能直接执行的补词单。一个 2 MB 的文件，断网可用，数据只在本机。"

`2 分钟版`

"先讲测准：分层抽样 + 三层修正 + 目标精度自适应停测，题量由你要的精度决定。再讲诊断：总掌握率会骗人，所以拆到七个词频段，按「缺口 × 词频权重」排出先补哪段。最后讲行动：补词单具体到词、按你每天的时间折算天数、隔一段回来复测验证。AI 只做解释和排序，不负责算数。"

`被问住时的接法（不要硬编）`

"这一点我目前没有数据支撑 / 没做到，我的理解是……（说清边界）。如果要补，我会先做（具体一步）。" —— 承认边界比编答案得分高，尤其是碰到信效度、并发、安全这类深水区。

6.4 现场应急清单

断网怎么办	本地所有功能照常；AI 面板会自动回落到"本地方案"并写明原因。提前把这条当卖点讲，而不是当事故。
AI 面板打不开	先确认档位徽标（自带 Key / 作者额度 / 本地方案）；连不上就直接讲"现在没有网络，这是本地方案的诊断"，顺势演示确定性规则。
数据是空的	用设置页「一键生成六个等级的演示账号」，切到 S/A/B+/B/C/D 任意一档演示；生成的是真实作答数据，等级由引擎现算，不是贴图。
被问到不会的	用 6.3 的"承认边界 + 说清理解 + 下一步"。别编术语。
时间不够	优先讲：定位 → 三个亮点 → 一个现场演示（点频段条问 AI）→ 一句收尾。其余都是可跳页。

配套材料：技术文档（22 页 PDF，公式与实现细节）、产品介绍（10 章）、功能总览、19 篇开发报告（每篇一个真实缺陷的复盘）、NEXT.md（设计决策与试过的错路）。